BLOCKSEC

Security Audit

Report for multisig-wallet-contracts

Date: July 13, 2026 **Version:** 1.0

Contact: contact@blocksec.com

Contents

Report Manifest

Item	Description
Client	KEYRING
Target	multisig-wallet-contracts

Version History

Version	Date	Description
1.0	July 13, 2026	First release

Signature

Digitally signed by: BlockSec (https://blocksec.com)

SHA1 digest of public key: BAB106D3708BC6DE0963CE05E9478C24B326AE1C

Date: 2026-07-13 05:48:18+0000

About BlockSec BlockSec focuses on the security of the blockchain ecosystem and collaborates with leading DeFi projects to secure their products. BlockSec is founded by top-notch security researchers and experienced experts from both academia and industry. They have published multiple blockchain security papers in prestigious conferences, reported several zero-day attacks of DeFi applications, and successfully protected digital assets that are worth more than 14 million dollars by blocking multiple attacks. They can be reached at Email, Twitter and Medium.

Chapter 1 Introduction

1.1 About the Audit Target

Information	Description
Type	Smart Contract
Language	Solidity
Approach	Semi-automatic and manual verification

The audit target (hereinafter referred to as the Target) of this audit is the code repository [1] of multisig-wallet-contracts of KEYRING.

This project implements a permissionless multisig wallet system. A factory deploys deterministic minimal-proxy wallets for users and stores each wallet's signer set and approval threshold. Wallets can receive native tokens, ERC20, ERC721, and ERC1155 assets, and allow withdrawals only after validating threshold EIP-712 signatures from approved signers. Each withdrawal type uses an independent nonce and deadline for replay protection, while recipients are restricted to wallet signers. The factory also emits standardized withdrawal events for created wallets.

Note this audit only focuses on the smart contracts in the following directories/files:

- src/

Other files are not within the scope of the audit. Additionally, all dependencies of the Target are considered reliable in terms of both functionality and security, and are therefore not included in the audit scope.

The auditing process is iterative. Specifically, we would audit the commits that fix the discovered issues. If there are new issues, we will continue this process. The commit SHA values during the audit are shown in the following table. Our audit report is responsible for the code in the initial version (`Version 1`), as well as new code (in the following versions) to fix issues in the audit report. Code prior to and including the baseline version (`Version 0`), where applicable, is outside the scope of this audit and assumes to be reliable and secure.

Project	Version	Commit Hash
multisig-wallet-contracts	`Version 1`	`4a96b51a3c98a6f56a94d252d04195247b5ebf8a`
	`Version 2`	`48089ed1a2e516b2d79c333200acccf5ea4b84de`

1.2 Disclaimer

This audit report does not constitute investment advice or a personal recommendation. It does not consider, and should not be interpreted as considering or having any bearing on, the potential economics of a token, token sale or any other product, service or other asset. Any entity should not rely on this report in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset.

[1] https://github.com/keyring-one/multisig-wallet-contracts

This audit report is not an endorsement of any particular project or team, and the report does not guarantee the security of any particular project. This audit does not give any warranties on discovering all security issues of the Target, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit cannot be considered comprehensive, we always recommend proceeding with independent audits and a public bug bounty program to ensure the security of the Target.

The scope of this audit is limited to the code mentioned in Section 1.1. Unless explicitly specified, the security of the language itself (e.g., the solidity language), the underlying compiling toolchain and the computing infrastructure are out of the scope.

1.3 Procedure of Auditing

We perform the audit according to the following procedure.

- **Vulnerability Detection** We first scan the Target with automatic code analyzers, and then manually verify (reject or confirm) the issues reported by them.
- **Semantic Analysis** We study the business logic of the Target and conduct further investigation on the possible vulnerabilities using an automatic fuzzing tool (developed by our research team). We also manually analyze possible attack scenarios with independent auditors to cross-check the result.
- **Recommendation** We provide some useful advice to developers from the perspective of good programming practice, including gas optimization, code style, and etc.

We show the main concrete checkpoints in the following.

1.3.1 Security Issues

* Access control
* Permission management
* Whitelist and blacklist mechanisms
* Initialization consistency
* Improper use of the proxy system
* Reentrancy
* Denial of Service (DoS)
* Untrusted external call and control flow
* Exception handling
* Data handling and flow
* Events operation
* Error-prone randomness
* Oracle security
* Business logic correctness
* Semantic and functional consistency
* Emergency mechanism
* Economic and incentive impact

1.3.2 Additional Recommendation

* Gas optimization
* Code quality and style

Note *The previous checkpoints are the main ones. We may use more checkpoints during the auditing process according to the functionality of the project.*

1.4 Security Model

To evaluate the risk, we follow the standards or suggestions that are widely adopted by both industry and academy, including OWASP Risk Rating Methodology [2] and Common Weakness Enumeration [3]. The overall *severity* of the risk is determined by *likelihood* and *impact*. Specifically, likelihood is used to estimate how likely a particular vulnerability can be uncovered and exploited by an attacker, while impact is used to measure the consequences of a successful exploit.

In this report, both likelihood and impact are categorized into two ratings, i.e., *high* and *low* respectively, and their combinations are shown in Table 1.1.

Table 1.1: Vulnerability Severity Classification

Impact \ Likelihood	*High*	*Low*
High	High	Medium
Low	Medium	Low

Accordingly, the severity measured in this report are classified into three categories: **High**, **Medium**, **Low**. For the sake of completeness, **Undetermined** is also used to cover circumstances when the risk cannot be well determined.

Furthermore, the status of a discovered item will fall into one of the following five categories:

- **Undetermined** No response yet.
- **Acknowledged** The item has been received by the client, but not confirmed yet.
- **Confirmed** The item has been recognized by the client, but not fixed yet.
- **Partially Fixed** The item has been confirmed and partially fixed by the client.
- **Fixed** The item has been confirmed and fixed by the client.

[2] https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

[3] https://cwe.mitre.org/

Chapter 2 Findings

In total, we found **one** potential security issue. Besides, we have **three** recommendations.

- Low Risk: 1
- Recommendation: 3

ID	Severity	Description	Category	Status
1	Low	Lack of EOA signer validation in function `createMultisigWallet()`	Security Issue	Fixed
2	-	Add zero address validation in function `constructor()`	Recommendation	Fixed
3	-	Enforce a dynamic minimum signature threshold	Recommendation	Fixed
4	-	Emit only verified signatures in withdrawal events	Recommendation	Fixed

The details are provided in the following sections.

2.1 Security Issue

2.1.1 Lack of EOA signer validation in function `createMultisigWallet()`

Severity Low

Status Fixed in `Version 2`

Introduced by `Version 1`

Description The function `createMultisigWallet()` allows arbitrary addresses to be configured as wallet signers without distinguishing between externally owned accounts (EOAs) and contract accounts. However, withdrawal authorization relies exclusively on function `ECDSA.recover()` to recover signer addresses from EIP-712 signatures.

Since function `ECDSA.recover()` only supports signatures produced by EOAs, contract accounts cannot be validated through the current signature verification flow. As a result, if one or more contract addresses are registered as signers and the configured signature threshold depends on them, the required number of valid signatures may become unattainable, preventing authorized withdrawals.

```
83   function createMultisigWallet(uint256 id_, uint256 signerThreshold_, address[] calldata
         signers_) external {
84       address user = msg.sender;
85
86       if (signerThreshold_ < MIN_SIGNER_THRESHOLD) {
87           revert InvalidSignerThreshold();
88       }
89       if (signers_.length < signerThreshold_) {
90           revert InvalidSigners();
91       }
92       address lastSigner = address(0);
93       for (uint256 i = 0; i < signers_.length; i++) {
```

```
94          if (signers_[i] <= lastSigner) {
95              revert InvalidSortedSigners();
96          }
97
98          lastSigner = signers_[i];
99      }
100
101     address multisigWallet =
102         Clones.cloneDeterministic(multisigWalletImplementation, keccak256(abi.encode(user, id_)
                ));
103     MultisigWallet(payable(multisigWallet)).initialize(address(this));
104
105     multisigWallets[multisigWallet] = true;
106
107     signerThreshold[multisigWallet] = signerThreshold_;
108     for (uint256 i = 0; i < signers_.length; i++) {
109         _signers[multisigWallet].add(signers_[i]);
110     }
111
112     emit MultisigWalletCreated(multisigWallet, user, id_, signerThreshold_, signers_);
113 }
```

Listing 2.1: src/MultisigWalletFactory.sol

Impact If contract addresses are registered as signers, the wallet may become unable to satisfy the required signature threshold, potentially preventing legitimate withdrawals and causing assets to remain inaccessible.

Suggestion Add a check to ensure that all multisig wallet signers are EOAs.

2.2 Recommendation

2.2.1 Add zero address validation in function `constructor()`

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In the `MultisigWalletFactory` contract, the function `constructor()` is used to initialize the immutable `multisigWalletImplementation` address used for deterministic wallet deployment. However, it does not validate that the provided implementation address is non-zero.

Since the implementation address is immutable and cannot be updated after deployment, supplying the zero address during deployment would permanently prevent the factory from deploying new multisig wallet instances.

```
67  constructor(address multisigWalletImplementation_) {
68      multisigWalletImplementation = multisigWalletImplementation_;
69  }
```

Listing 2.2: src/MultisigWalletFactory.sol

Suggestion Add a zero address check in the function `constructor()`.

2.2.2 Enforce a dynamic minimum signature threshold

Status Fixed in `Version 2`

Introduced by `Version 1`

Description The function `createMultisigWallet()` validates that the configured signature threshold is not less than the fixed constant `MIN_SIGNER_THRESHOLD`, which is currently set to 2.

As a result, the minimum acceptable threshold remains the same regardless of the number of configured signers. For example, a wallet with a large signer set may still be created with a threshold of only two signatures. Depending on the intended security model, this may provide a lower level of authorization than expected for wallets with many signers.

```
86        if (signerThreshold_ < MIN_SIGNER_THRESHOLD) {
87            revert InvalidSignerThreshold();
88        }
```

Listing 2.3: src/MultisigWalletFactory.sol

Suggestion Add dynamic minimum threshold validation based on `signers_.length`, for example using a 2/3 majority threshold.

2.2.3 Emit only verified signatures in withdrawal events

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In the contract `MultisigWallet`, each withdrawal function validates `signatures_` through function `_validateSignatures()` before executing the withdrawal, and subsequently emits the original `signatures_` array in the corresponding withdrawal event. However, function `_validateSignatures()` returns immediately once the required signature threshold has been reached. As a result, any remaining signatures in the array are not processed as part of the authorization flow.

Consequently, withdrawal events may include signatures that were not involved in satisfying the required approval threshold. Off-chain systems consuming these events may incorrectly interpret all emitted signatures as having participated in authorizing the withdrawal.

```
300    function _validateSignatures(bytes32 hash_, Types.Signature[] calldata signatures_) internal
           view {
301        uint256 signerThreshold = multisigWalletFactory.signerThreshold(address(this));
302
303        uint256 validSignatures;
304        address lastAddress;
305
306        bytes32 digest = _hashTypedDataV4(hash_);
307
308        for (uint256 i = 0; i < signatures_.length; i++) {
309            address recovered = ECDSA.recover(digest, signatures_[i].v, signatures_[i].r,
                   signatures_[i].s);
310
```

```
311             if (recovered <= lastAddress) {
312                 revert InvalidSignatureOrder();
313             }
314
315             if (multisigWalletFactory.isSigner(address(this), recovered)) {
316                 validSignatures++;
317
318                 if (validSignatures == signerThreshold) {
319                     return;
320                 }
321             }
322
323             lastAddress = recovered;
324         }
325
326         revert InvalidSignatures();
327     }
```

Listing 2.4: src/MultisigWallet.sol

Suggestion Revise the event emission logic to emit only the signatures that were processed during authorization.


BOOST WEB3 THROUGH NEXT-GENERATION SECURITY & USABILITY INNOVATIONS